

Collecting data over HTTP and APIs

10,000+ movies for the Netflix project, fetched one HTTP request at a time from APIs such as OMDb.

THE QUESTION

How does a Python script ask a server for movie data, and know the answer is usable?

THE INTUITION

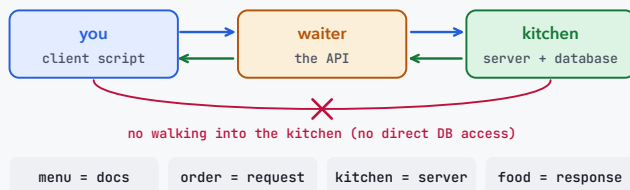
An API is a waiter: you order from the menu (docs), the kitchen (server) cooks, the food (response) comes back. In HTTP terms, a request goes out; a status code, headers and JSON come back.

KEY TAKEAWAY

Check the status code before the body, put a timeout on every call, and back off on 429 or 5xx.

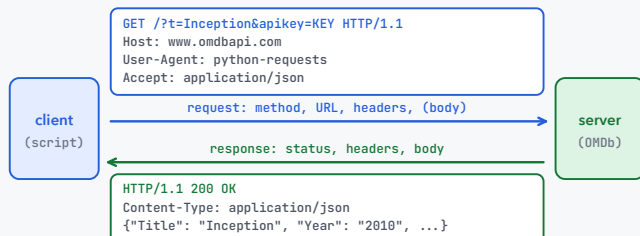
DevTools → curl: test → requests: automate → check status + JSON → save raw JSON → DataFrame → CSV

1 An API is a controlled window onto data



Like a bank teller, it exposes only what the owner chooses, validates each request, logs usage and rate-limits. Read the docs for the **base URL**, **authentication**, **endpoints** and **rate limits**. Prefer a ready dataset, then an API, then scraping (see the scraping sheet).

2 One request out, one response back



HTTP is plain text: see it in the DevTools Network tab, then "Copy as cURL". It is **stateless**, like a goldfish, so the key or token travels with every call. Port 80 is HTTP, 443 is HTTPS. Headers are labels: Accept asks for a format, Content-Type names the body's format, Authorization carries a key.

3 A URL has four parts

https://www.omdbapi.com/?t=Inception&y=2010&apikey=KEY

title year auth

scheme host path query

port 443 is implied by https · pairs joined by &

```
params={"s": "Star Wars", "type": "movie"}
requests encodes it as
?s=Star+Wars&type=movie (space became +)
```

Pass query parameters as a `params=` dict and let requests encode them; never glue the string by hand; `r.url` shows the URL it built. In a shell, quote any URL containing ? or &.

4 GET reads, POST sends

GET /movies?t=Inception
params ride in the URL
reads data (~90% of calls)

POST /items + JSON body
{"name": "Mango", ...}
sends data, 201 Created

GET /movies
GET /movies/123
POST /movies
PUT /movies/123
DELETE /movies/123

→ list all
→ one movie
→ create
→ update
→ delete

REST: URLs are nouns, methods are verbs

Data collection is almost all GET. POST carries data in the **body**: in requests, `json=` sends JSON (`application/json`) and `data=` sends a form (`application/x-www-form-urlencoded`).

5 The first digit says who failed

code	examples	what your script does
2XX success	200 OK · 201 Created	parse the body
3XX redirect	301 Moved · 302 Found	follow (requests does)
4XX your fault	400 · 401 · 403 · 404	fix the request
4XX too fast	429 Too Many Requests	wait, then retry
5XX their fault	500 · 503	retry later

401: missing or bad key. 403: not allowed. 404: no such resource. A 5xx or 429 can clear up if you wait; any other 4xx fails again until you change the request.

```
if r.status_code == 200: data = r.json()
elif r.status_code == 429: time.sleep(60)
elif r.status_code >= 500: time.sleep(5)
```

6 JSON arrives as text with loose types

```
{
  "Title": "Inception",
  "Year": "2010",
  "Rated": "PG-13",
  "Director": "Christopher Nolan",
  "imdbRating": "8.8",
  "imdbID": "tt1375666",
  "Response": "True"
}
```

numbers arrive as text:

```
int(d["Year"]) → 2010
```

a missing key crashes:

```
d.get("Director", "Unknown")
```

null becomes None;

```
no date type: "2010-07-16"
```

JSON has six value types: string, number, boolean, null, array, object. OMDb sends every value as a string. "", null and a missing key are three different cases.

Collect politely, fail loudly

The same call from the terminal and from Python, then keys, quotas, retries and error handling for a real crawl.

7 curl and requests make the same call

Test in the terminal, then automate in Python. Both fetch Inception from OMDb.

```
URL="https://www.omdbapi.com/"
curl -s "$URL?t=Inception&apikey=$KEY" | jq .
```

```
r = requests.get(URL, timeout=10,
    params={"t": "Inception", "apikey": KEY})
r.status_code      # 200
r.json()["Year"]    # '2010', a string
```

curl flags: -X method, -H header, -d body, -o save to file, -I headers only, -v show both sides, -L follow redirects, --max-time timeout. curl needs jq for JSON and signals errors by exit code; requests gives .json(), loops and exceptions.

8 Keys identify you; keep them out of code

1 in the URL	?apikey=KEY	OMDb
2 in a header	X-API-Key: KEY	
3 bearer token	Authorization: Bearer TOKEN	OAuth

Keys let the provider track usage, enforce limits and bill. OMDb reads apikey from the query (free tier: 1,000 requests a day) plus t title, i IMDb ID, y year, s search, page. Load the key from the environment: `os.environ["OMDB_API_KEY"]`.

11 Loop, collect, keep the raw JSON

```
rows = []
for title in ["Inception", "Avatar"]:
    d = get_movie(title)      # panel 10
    if d: rows.append(d)      # full JSON
    time.sleep(0.5)          # be polite
df = pd.DataFrame(rows)
df.to_csv("movies.csv", index=False)
```

Save full responses so a missed field never means re-collecting 10,000 movies. Log failures with reasons and checkpoint so a crash resumes. Cleaning comes next, on the data validation sheet.

12 Pause and think

OMDb says 200 for a film that does not exist. Why? The HTTP call worked; "Response": "False" in the JSON reports the failed lookup. Stateless, so how do you stay logged in? Send the key or token with every request.

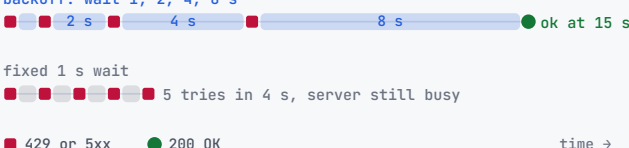
Why params= over string building? requests encodes spaces and symbols, so "Star Wars" arrives intact.

What does `raise_for_status()` add? A 4xx or 5xx becomes an exception you cannot miss.

Should you retry a 404? No, the same request fails again. Retry 429 and 5xx; fix a 4xx.

10,000 movies on the free tier? At least 10 days, which is why raw responses get saved.

9 Pace the crawl, back off on failure

backoff: wait 1, 2, 4, 8 s


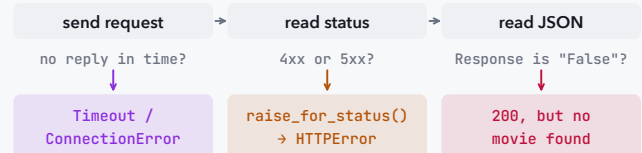
`time.sleep(1)` between calls is the simple fix. After a 429 or 5xx, wait 2^{attempt} seconds and try again. The response header X-RateLimit-Remaining shows what is left. Retry only 429, 5xx and timeouts.

10,000 movies ÷ 1,000 requests/day = 10 days on the free tier

day 1 day 2 day 3 day 4 day 5 day 6 day 7 day 8 day 9 day 10

with `sleep(0.5)`, one day's 1,000 calls take only ~8 min: the daily quota, not the delay, sets the pace

10 A 200 does not mean you got the movie



```
def get_movie(title):
    p = {"t": title, "apikey": KEY}
    try:
        r = requests.get(URL, params=p,
            timeout=10)
        r.raise_for_status() # 4xx/5xx raise
        d = r.json()
    except requests.Timeout:
        return None # retry later
    except requests.RequestException as e:
        print("failed:", title, e)
        return None
    if d.get("Response") != "True":
        return None # 200, no movie
    return d
```

Without `timeout=`, requests can wait forever on a stalled server.

status first, then body · params dict, not string building · timeout on every call · back off on 429 and 5xx

Source: lecture "Data collection for machine learning" (HTTP and API parts) with its demos and lab, nipunbatra.github.io/stt-ai-teaching. HTML parsing and scraping are on the scraping sheet.